

# Excel Name Comparer

## Technical Documentation

**Version:** 1.0

**Author:** Orlando Caetano

**Language:** Python 3

**Development Environment:** Visual Studio Code (VS Code)

**Operating System:** macOS (Apple Silicon M4 Pro)

**Source Control:** Git & GitHub

---

## 1. Project Overview

### Purpose

The Excel Name Comparer application was developed to compare names contained in two separate Excel spreadsheets and identify records that do not exist in both files.

The application reads two Excel files, normalizes the names to ensure consistent formatting, compares the datasets, and generates a new Excel file containing only the unmatched names.

This tool is intended to reduce the time required to manually compare lists of names from different sources and improve accuracy when working with large datasets.

---

## 2. Business Requirement

The client required a solution capable of:

- Reading names from two Excel spreadsheets.
  - Handling different spreadsheet structures.
  - Normalizing names before comparison.
  - Detecting names that appear in only one of the spreadsheets.
  - Producing a new Excel file containing unmatched names.
  - Providing a maintainable and testable codebase.
  - Using version control through Git and GitHub.
- 

## 3. Functional Requirements

The application shall:

1. Read data from Excel files.
2. Support a single "Full Name" column.
3. Support separate "First Name" and "Surname" columns.
4. Normalize name formatting.
5. Compare names accurately.
6. Generate an output Excel file.
7. Handle missing files appropriately.
8. Handle missing columns appropriately.
9. Provide command-line execution.
10. Be fully tested using pytest.

---

## 4. Technology Stack

| Component            | Technology         |
|----------------------|--------------------|
| Programming Language | Python 3           |
| IDE                  | Visual Studio Code |
| Data Processing      | Pandas             |
| Excel Support        | OpenPyXL           |
| Testing Framework    | Pytest             |
| Version Control      | Git                |
| Repository Hosting   | GitHub             |

---

## 5. Project Structure

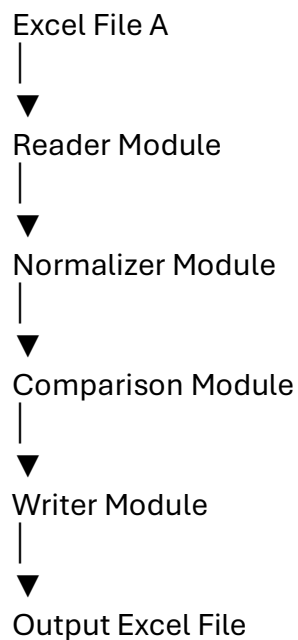
```
excel-name-comparer/  
|  
├── src/  
│   ├── __init__.py  
│   ├── reader.py  
│   ├── normalizer.py  
│   ├── comparer.py  
│   ├── writer.py  
│   └── config.py  
├── tests/  
│   ├── test_reader.py  
│   ├── test_normalizer.py  
│   ├── test_comparer.py  
│   ├── test_writer.py  
│   └── test_integration.py
```

```
├── data/
│   ├── input/
│   └── output/
├── main.py
├── requirements.txt
├── README.md
└── .gitignore
```

---

## 6. System Architecture

The application follows a modular architecture where each module has a single responsibility.



## 7. Module Descriptions

### 7.1 Reader Module

#### File

src/reader.py

#### Responsibilities

- Read Excel spreadsheets.
- Validate file existence.
- Validate column names.

- Extract names from specified columns.
- Support both single-column and multi-column formats.

### Example

Input:

| First Name | Surname |
|------------|---------|
| John       | Smith   |

Output:

Python  
["John Smith"]

---

## 7.2 Normalizer Module

### File

src/normalizer.py

### Responsibilities

Standardise names before comparison.

The module performs:

- Conversion to lowercase.
- Removal of leading spaces.
- Removal of trailing spaces.
- Removal of multiple spaces.
- Removal of titles.
- Conversion of surname-first formats.
- Removal of special characters and accents.

### Example

Input:

Caetano, ORLANDO (Mr)

Output:

orlando caetano

---

## 7.3 Comparison Module

### File

src/comparer.py

### Responsibilities

Compare normalized names and identify records that do not exist in both datasets.

### Method Used

Python set operations:

```
Python  
set_a.symmetric_difference(set_b)
```

### Example

List A

```
Python  
["john smith", "mary jones"]
```

List B

```
Python  
["mary jones", "sarah green"]
```

Output

```
Python  
["john smith", "sarah green"]
```

---

## 7.4 Writer Module

### File

src/writer.py

### Responsibilities

Generate an output Excel spreadsheet containing unmatched names.

### Output Example

| Name        |
|-------------|
| john smith  |
| sarah green |

---

## 7.5 Configuration Module

### File

src/config.py

### Responsibilities

Process command-line arguments using argparse.

Allows users to specify:

- Input files
- Column names
- Runtime configuration

without modifying application source code.

---

## 8. Error Handling

The application includes validation for:

### Missing Files

Example:

Plain Text  
FileNotFoundError

Raised when an Excel file cannot be found.

---

### Missing Columns

Example:

ValueError

Raised when the requested column does not exist within an Excel file.

---

### **Empty Data**

The application safely handles empty datasets and returns empty comparison results when appropriate.

---

## **9. Testing Strategy**

### **Framework**

pytest

### **Types of Testing**

#### **Unit Testing**

Each module is tested independently.

Examples:

- Reader tests
  - Normalizer tests
  - Comparator tests
  - Writer tests
- 

#### **Integration Testing**

Tests the complete business process:

Read

↓

Normalize

↓

Compare

↓

Export

---

### **Running Tests**

Shell  
pytest

Verbose output:

Shell  
pytest -s

---

## 10. Installation

### Clone Repository

Shell  
git clone https://github.com/<OMTPC>

Navigate to project folder:

Shell  
cd excel-name-comparer

---

### Create Virtual Environment

Shell  
python3 -m venv venv

Activate environment:

Shell  
source venv/bin/activate

---

### Install Dependencies

Shell  
pip install -r requirements.txt

---

## 11. Application Usage

Example execution:

Shell  
python3 main.py \  
data/input/file\_a.xlsx \  
data/input/file\_b.xlsx \

```
--columns-a Name \  
--columns-b "First Name" Surname
```

---

## Output

Example console output:

```
Loaded 156 names from first file.  
Loaded 149 names from second file.  
Found 11 unmatched names.  
Output written to:
```

```
data/output/unmatched_names.xlsx
```

---

## 12. Source Control

### Git

The project uses Git for version control.

Typical workflow:

```
Shell  
git add .  
git commit -m "Description of change"  
git push
```

---

### GitHub

The project is hosted in a GitHub repository for:

- Source management
  - Backup
  - Version history
  - Collaboration
- 

## 13. Future Improvements

Potential future enhancements include:

### Fuzzy Matching

Identify similar names such as:

Jon Smith  
John Smith

using libraries such as:

RapidFuzz

---

## Graphical User Interface

Create a desktop interface using:

Tkinter

or

PySide6

to allow non-technical users to select files through a graphical interface.

---

## Enhanced Reporting

Generate output containing:

| Name        | Source |
|-------------|--------|
| John Smith  | File A |
| Sarah Green | File B |

This would identify the origin of each unmatched record.

---

## 14. Conclusion

The Excel Name Comparer application demonstrates the use of modern software engineering principles, including modular design, automated testing, version control, error handling, and separation of concerns.

The solution provides a reliable and maintainable approach for comparing Excel-based name datasets while remaining extensible for future enhancements and client requirements.

---

**End of Document**